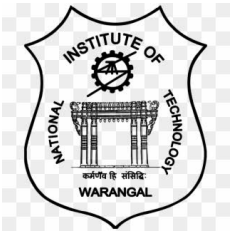
National Institute of Technology, Warangal

Computer Science and Engineering



Human Activity Recognition

Submitted in partial fulfilment of the requirements for

Data Science Fundamentals Course

Master's in Technology

Submitted by: Hrishikesh Chandra
Student ID: 26CSM1R05
Programme: M.tech CSE
Semester: 1
Academic Year: 2026-2028

Under the guidance of

Prof. Pisipati Radha Krishna

Declaration

I hereby declare that the project report entitled **"Human Activity RECOGNITION"** is a record of the work carried out by me as part of the DSF project, under the guidance of Prof. Pisipati Radha Krishna. The implementation details and experimental results presented in this report are based on the submitted project notebook and associated dataset files.

Place: NIT Campus
Date: 07/Nov/2026

Signature: Hrishikesh Chandra

Acknowledgement

I express my sincere gratitude to Prof. Pisipati Radha Krishna for the guidance and support provided during this project. I also thank the Department CSE, NIT Warangal for providing the academic environment and resources required to complete the work. Finally, I thank my peers and everyone who contributed directly or indirectly to the successful completion of this project.

Abstract

Human Activity Recognition (HAR) is the task of identifying a person's physical activity from sensor measurements. This project develops a machine-learning workflow to classify six activities: walking, walking upstairs, walking downstairs, sitting, standing and laying. The supplied dataset contains 561 engineered features for each observation, with 7,352 training samples and 2,947 test samples. Three classifiers are trained and evaluated: K-Nearest Neighbours (KNN), Decision Tree and Random Forest. KNN uses a standardisation step within a machine-learning pipeline, while the Decision Tree and Random Forest are trained directly on the supplied feature vectors. Performance is evaluated using accuracy, macro-averaged precision, recall and F1-score, along with class-wise classification reports and confusion matrices. *Random Forest obtains the strongest test performance, with 92.87% accuracy and a macro F1-score of 92.70%.* The trained models and supporting metadata are also exported for use by a Streamlit web application. The report documents the dataset, workflow, implementation, evaluation and application deployment evidence available from the project files.

Keywords: Human Activity Recognition, Machine Learning, KNN, Decision Tree, Random Forest, Sensor Features, Classification.

Contents

Declaration i

Acknowledgement ii

Abstract iii

1 Introduction and Problem Statement 1

1.1 Introduction 1

1.2 Problem Statement 1

1.3 Motivation 2

1.4 Project Objectives 2

1.5 Scope 3

1.6 Expected Outcome 3

2 Dataset Description and System Architecture 4

2.1 Dataset Overview 4

2.2 Activity Labels 5

2.3 Feature Description 5

2.4 Data Quality Checks 5

2.5 Activity Distribution 6

2.6 Proposed System Architecture 6

2.7 Software and Libraries 7

3 Methodology and Model Development 8

3.1 Overall Workflow 8

3.2 Data Loading and Label Mapping 8

3.3 Preprocessing and Data Preparation 8
3.3.1 Feature Distribution 9
3.4 K-Nearest Neighbours (KNN) 9
3.5 Decision Tree Classifier 10
3.6 Random Forest Classifier 11
3.7 Evaluation Metrics 11
3.8 Best Model Selection 12
3.9 Saving and Exporting Models 12

4 Experimental Results and Analysis **13**
4.1 Overall Model Performance 13
4.2 Model Comparison Graph 13
4.3 Confusion Matrix Analysis 14
4.4 Class-wise Evaluation 17
4.4.1 Class-wise report: KNN 18
4.4.2 Class-wise report: Decision Tree 18
4.4.3 Class-wise report: Random Forest 18
4.5 Feature Importance 19
4.6 Single-Observation Prediction 20
4.7 Best Model 20

5 Web Application, Discussion and Conclusion **21**
5.1 Streamlit Web Application 21
5.1.1 Application Screenshots 22
5.2 Discussion 23
5.3 Limitations 23
5.4 Future Enhancements 24
5.5 Conclusion 24

References **25**

List of Figures

2.1 Activity distribution in the training and testing datasets. 6

2.2 High-level architecture of the HAR machine-learning workflow. 7

3.1 Distributions of four selected input features. 9

4.1 Accuracy and macro F1-score comparison. 14

4.2 Confusion matrix for KNN. 15

4.3 Confusion matrix for Decision Tree. 16

4.4 Confusion matrix for Random Forest. 17

4.5 Top 20 features ranked by Random Forest feature importance. 19

5.1 Streamlit application home page. 22

5.2 Example activity prediction in the web application. 22

List of Tables

2.1 Dataset dimensions used in the project 4

2.2 Activity identifiers and their meanings 5

4.1 Overall test-set performance comparison 13

4.2 Class-wise precision, recall and F1-score for KNN 18

4.3 Class-wise precision, recall and F1-score for Decision Tree 18

4.4 Class-wise precision, recall and F1-score for Random Forest 18

4.5 Top 10 Random Forest features by importance 20

5.1 Files exported for the web application 21

Chapter 1

Introduction and Problem Statement

1.1 Introduction

Human Activity Recognition (HAR) refers to the automatic identification of human physical activities using measurements collected from sensors. Wearable and smartphone devices commonly contain inertial sensors such as accelerometers and gyroscopes. These sensors capture movement-related information along multiple axes. By analysing the resulting signals, a machine-learning system can infer the activity being performed.

HAR has applications in health and wellness monitoring, assisted living, fitness tracking, context-aware mobile applications, rehabilitation support and human-computer interaction. A reliable activity recognition system can convert low-level sensor measurements into meaningful activity labels that are easier for an application or user to interpret.

This project focuses on a supervised classification approach. Instead of processing raw sensor streams directly in the notebook, the implementation uses a prepared feature dataset. Each observation is represented by 561 numerical features derived from time-domain and frequency-domain sensor signal characteristics. The machine-learning models learn the relationship between these features and the corresponding activity labels.

1.2 Problem Statement

The problem addressed in this project is to develop and evaluate a machine-learning system that can classify a human activity into one of six predefined categories from a numerical feature vector obtained from inertial sensor measurements.

The six target activities are:

1. Walking

2. Walking upstairs
3. Walking downstairs
4. Sitting
5. Standing
6. Laying

The system must learn from labelled training observations and predict the activity label for previously unseen test observations. Since several activities can produce similar sensor patterns, particularly the static postures sitting, standing and laying, and the dynamic activities involving walking, the classification task requires a model capable of learning relationships across a relatively large feature space.

1.3 Motivation

Manual interpretation of multi-axis accelerometer and gyroscope signals is difficult and does not scale well for continuous monitoring. Machine learning provides a systematic way to identify patterns in engineered sensor features. Comparing multiple algorithms also helps determine which classifier offers the best balance of predictive performance for this dataset.

1.4 Project Objectives

- To load and understand the prepared HAR feature dataset.
- To associate numerical activity identifiers with their corresponding activity names.
- To inspect the data for missing values, duplicate rows and invalid numerical values.
- To train KNN, Decision Tree and Random Forest classifiers.
- To evaluate the classifiers using accuracy, precision, recall and macro F1-score.
- To analyse class-wise predictions using confusion matrices and classification reports.
- To identify influential features using Random Forest feature importance.
- To export the trained models and metadata for use in a Streamlit-based web application.

1.5 Scope

The present implementation is a software-based, supervised classification workflow using the supplied train/test feature files. It demonstrates model development, evaluation, comparison and model export. The notebook does not acquire live sensor signals, perform raw-signal filtering from sensor hardware, or demonstrate on-device inference. Those activities are outside the scope of the evidence supplied for this report.

1.6 Expected Outcome

The expected outcome is a trained classifier that predicts one of the six activity categories from a 561-dimensional feature vector. The three classifiers are compared on the same test set, and the model with the highest macro F1-score is selected as the best-performing model. The notebook results identify Random Forest as the best model.

Chapter 2

Dataset Description and System Architecture

2.1 Dataset Overview

The project uses a preprocessed Human Activity Recognition dataset containing feature vectors and activity labels. The files supplied for the project include the training and testing feature matrices (`X_train.txt`, `X_test.txt`), the corresponding target labels (`y_train.txt`, `y_test.txt`), a feature-name file and an activity-label mapping file.

Table 2.1: Dataset dimensions used in the project

Dataset component	**Number of records / features**
Training observations	7,352
Testing observations	2,947
Total observations	10,299
Input features per observation	561
Activity classes	6

The training matrix therefore has dimensions 7352×561*, while the test matrix has dimensions* 2947×561. The target vectors contain one integer activity identifier for each corresponding observation.

2.2 Activity Labels

Table 2.2: Activity identifiers and their meanings

Identifier	Activity
1	WALKING
2	WALKING_UPSTAIRS
3	WALKING_DOWNSTAIRS
4	SITTING
5	STANDING
6	LAYING

2.3 Feature Description

The supplied feature information describes variables derived from three-axis accelerometer and gyroscope signals sampled at 50 Hz. The documented signal processing includes median filtering and a third-order low-pass Butterworth filter with a 20 Hz corner frequency. Acceleration is separated into body acceleration and gravity acceleration using a low-pass Butterworth filter with a 0.3 Hz corner frequency. Jerk signals are derived from body acceleration and angular velocity. Signal magnitudes are calculated using the Euclidean norm, and Fast Fourier Transform (FFT) features are generated for selected signals.

The feature set includes statistical and signal descriptors such as mean, standard deviation, median absolute deviation, maximum, minimum, signal magnitude area, energy, interquartile range, entropy, autoregression coefficients, correlation, frequency-related descriptors and angles. The complete list of 561 feature names is provided in the supplied `features.txt` file.

Implementation clarification: These signal-processing operations are described in the supplied feature documentation as the origin of the prepared features. *The submitted notebook loads the already-computed 561-dimensional feature matrices. It does not independently apply the raw-signal filtering or FFT steps to sensor recordings.*

2.4 Data Quality Checks

The notebook checks the training and test feature matrices for missing values, duplicate rows and finite numerical values. The recorded results are:

- Missing values in training data: 0.

- Missing values in testing data: 0.
- Duplicate rows in training data: 0.
- Duplicate rows in testing data: 0.
- *All values in both matrices are finite.*

2.5 Activity Distribution

Figure 2.1 compares the number of samples in each class across the training and testing partitions. The training counts are 1,226 (walking), 1,073 (walking upstairs), 986 (walking downstairs), 1,286 (sitting), 1,374 (standing) and 1,407 (laying). The corresponding test counts are 496, 471, 420, 491, 532 and 537.

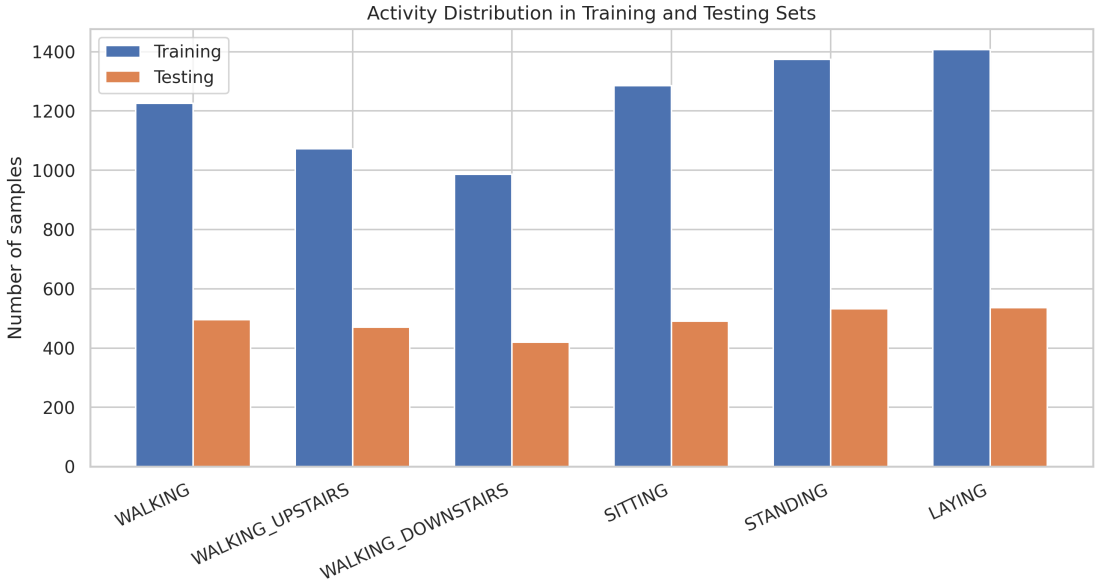


Figure 2.1: Activity distribution in the training and testing datasets.

2.6 Proposed System Architecture

The project workflow starts with the prepared HAR dataset and feature-label mapping. After loading and data-quality checks, the feature matrices are passed to three supervised classifiers. Predictions are evaluated using quantitative metrics and visual analysis. The trained models and metadata are exported as artifacts for the web application.

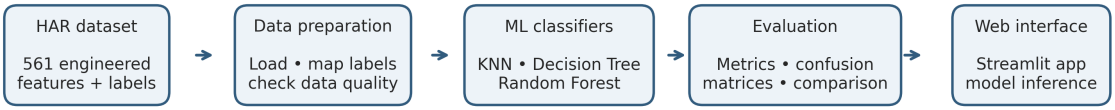


Figure 2.2: High-level architecture of the HAR machine-learning workflow.

2.7 Software and Libraries

The notebook uses Python and the following libraries: NumPy and Pandas for numerical and tabular data handling; Matplotlib and Seaborn for plots; Scikit-learn for preprocessing, model training and evaluation; and Joblib for saving trained models. The web application is hosted using Streamlit, according to the project URL supplied by the student.

Chapter 3

Methodology and Model Development

3.1 Overall Workflow

The notebook follows a sequence of data loading, feature and label preparation, data exploration, quality checking, model training, evaluation, comparison, model selection and artifact export. The original training and test partitions are retained. No additional random train-test split is created in the notebook.

3.2 Data Loading and Label Mapping

The feature-name file is read as a whitespace-separated text file. Duplicate feature names are made unique by adding suffixes to repeated names. The activity-label file maps integer class identifiers to the six activity names. The training and test feature matrices are then loaded and assigned the 561 feature names. The target vectors are loaded separately and mapped to readable activity names for exploration and plotting.

3.3 Preprocessing and Data Preparation

The preprocessing performed by the submitted notebook consists of the following steps:

1. Load the already-prepared training and test feature matrices.
2. Load the feature names and make duplicate names unique.
3. Load integer target labels and construct an activity-ID mapping.

4. Inspect the feature matrix dimensions and descriptive statistics.
5. Check for missing values, duplicated rows and non-finite values.
6. Copy the matrices and target vectors into the model-training variables.

The notebook reports zero missing values, zero duplicated rows and finite values throughout both feature matrices. The feature values are already in a normalised numerical range in the supplied dataset. The notebook does not apply a separate imputation procedure or a new feature-selection stage.

3.3.1 Feature Distribution

Figure 3.1 shows histograms for the first four features in the feature list. The distributions provide an initial view of the range and spread of selected sensor-derived variables.

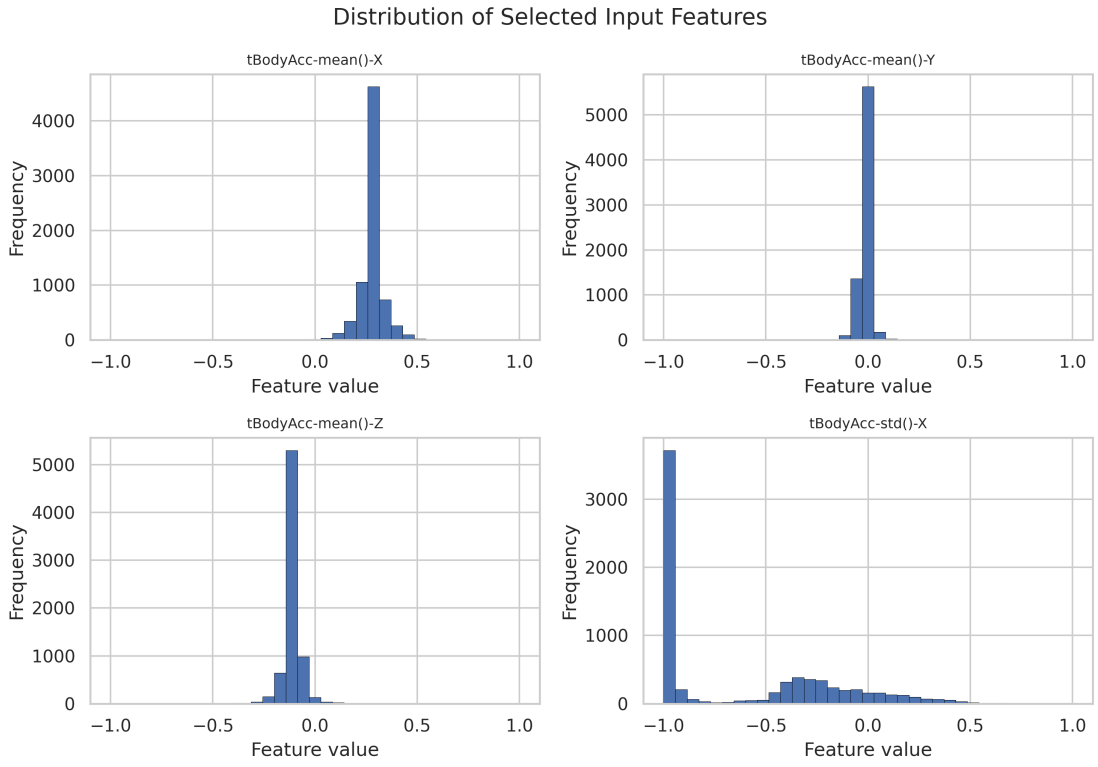


Figure 3.1: Distributions of four selected input features.

3.4 K-Nearest Neighbours (KNN)

KNN is a supervised, instance-based classification algorithm. For a new observation, it identifies the k closest training observations in feature space and assigns the class most frequently represented among those neighbours. In this project, $k = 5$.

The Euclidean distance between two feature vectors $\mathbf{x}$ and $\mathbf{z}$ is:

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{\sum_{j=1}^{p} (x_j - z_j)^2},$$

where $p = 561$ is the number of features. *Since distance-based methods can be affected by differences in feature scale, the implementation uses a Scikit-learn pipeline containing* `StandardScaler` *followed by* `KNeighborsClassifier(n_neighbors=5)`*. The scaler is fitted on the training data within the pipeline and then applied consistently during prediction.*

Listing 3.1: KNN model pipeline

```
knn_model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", KNeighborsClassifier(n_neighbors=5))
])
knn_model.fit(X_train_model, y_train_model)
knn_predictions = knn_model.predict(X_test_model)
```

3.5 Decision Tree Classifier

A Decision Tree learns a sequence of feature-based decision rules. At each internal node, the training data are divided according to a feature and threshold selected to improve class separation. A leaf node represents the predicted class.

The notebook uses Scikit-learn's `DecisionTreeClassifier` with `random_state=42`; other tree parameters are left at their library defaults. The model is fitted on the training feature matrix and used to predict the test observations.

Listing 3.2: Decision Tree training and prediction

```
decision_tree_model = DecisionTreeClassifier(
    random_state=42
)
decision_tree_model.fit(X_train_model, y_train_model)
decision_tree_predictions = decision_tree_model.predict(
    X_test_model
)
```

3.6 Random Forest Classifier

Random Forest is an ensemble-learning method that combines predictions from multiple decision trees. Each tree is trained using randomised samples and feature subsets. The final class is selected by aggregating the individual tree predictions, typically through majority voting.

The project configures the Random Forest with 200 estimators, a fixed random seed of 42 and parallel execution using `n_jobs=-1`. Other parameters retain their Scikit-learn defaults.

Listing 3.3: Random Forest training and prediction

```
random_forest_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)
random_forest_model.fit(X_train_model, y_train_model)
random_forest_predictions = random_forest_model.predict(
    X_test_model
)
```

3.7 Evaluation Metrics

The models are evaluated on the same 2,947 test observations. The following metrics are calculated:

- **Accuracy:** proportion of all test observations classified correctly.
- **Precision:** proportion of predicted examples of a class that are correct.
- **Recall:** proportion of actual examples of a class that are correctly identified.
- **F1-score:** harmonic mean of precision and recall.

For a multiclass problem, the notebook uses macro averaging for precision, recall and F1-score. Macro averaging calculates the metric separately for each class and then takes the unweighted mean:

$$\text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C} F1_c, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c},$$

where $C = 6$, P_c is the precision for class c and R_c is its recall.

3.8 Best Model Selection

The notebook selects the model with the highest macro F1-score. This criterion gives equal weight to each of the six activity classes. Based on the reported test results, Random Forest is selected with a macro F1-score of 0.9270.

3.9 Saving and Exporting Models

The notebook saves the three fitted estimators using Joblib. It also exports the test feature data, test labels, feature-importance values and a JSON metadata file. The metadata records feature names, activity mapping, evaluation results, selected best model and dataset dimensions. These files form the model artifacts used by the associated web application workflow.

Chapter 4

Experimental Results and Analysis

4.1 Overall Model Performance

All three classifiers are evaluated using the same held-out test set of 2,947 observations. Table 4.1 summarises the results. *Random Forest achieves the highest accuracy (92.87%), macro precision (93.00%), macro recall (92.58%) and macro F1-score (92.70%).* KNN ranks second in accuracy, while Decision Tree gives the lowest overall scores among the three evaluated models.

Table 4.1: Overall test-set performance comparison

Model	**Accuracy**	**Macro precision**	**Macro recall**	**Macro F1**
KNN	88.02%	88.97%	87.46%	87.66%
Decision Tree	86.22%	86.25%	85.87%	85.95%
Random Forest	92.87%	93.00%	92.58%	92.70%

4.2 Model Comparison Graph

Figure 4.1 compares accuracy and macro F1-score. Both measures show the same ranking: Random Forest, KNN and Decision Tree. The gap between Random Forest and KNN is approximately 4.85 percentage points in accuracy, while the gap between Random Forest and Decision Tree is approximately 6.65 percentage points.

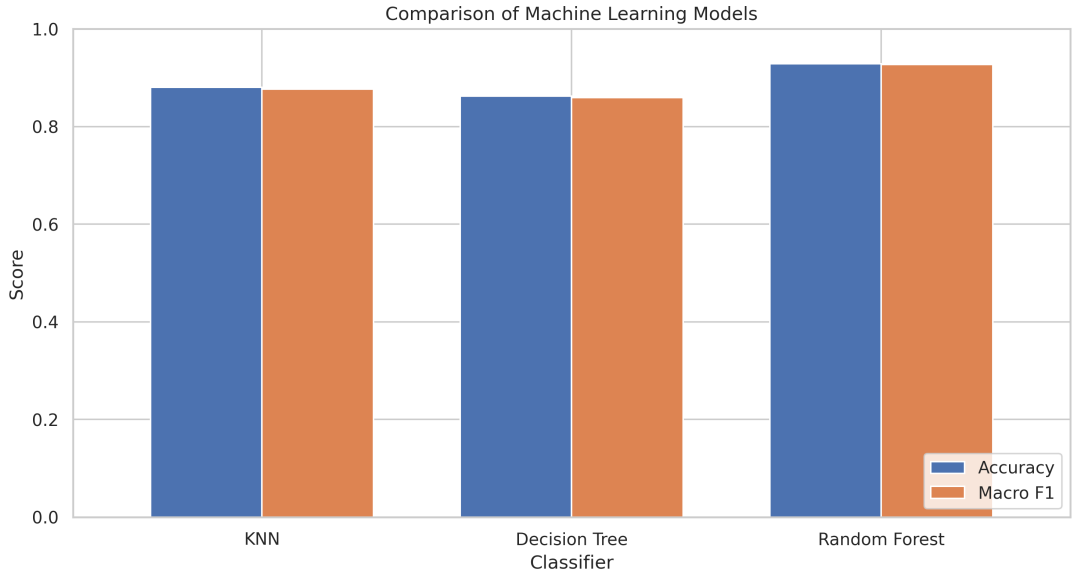


Figure 4.1: Accuracy and macro F1-score comparison.

4.3 Confusion Matrix Analysis

A confusion matrix compares actual class labels with predicted class labels. Correct predictions appear along the main diagonal, while off-diagonal values represent misclassifications. The row labels in the following figures correspond to actual activities and the column labels correspond to predicted activities.

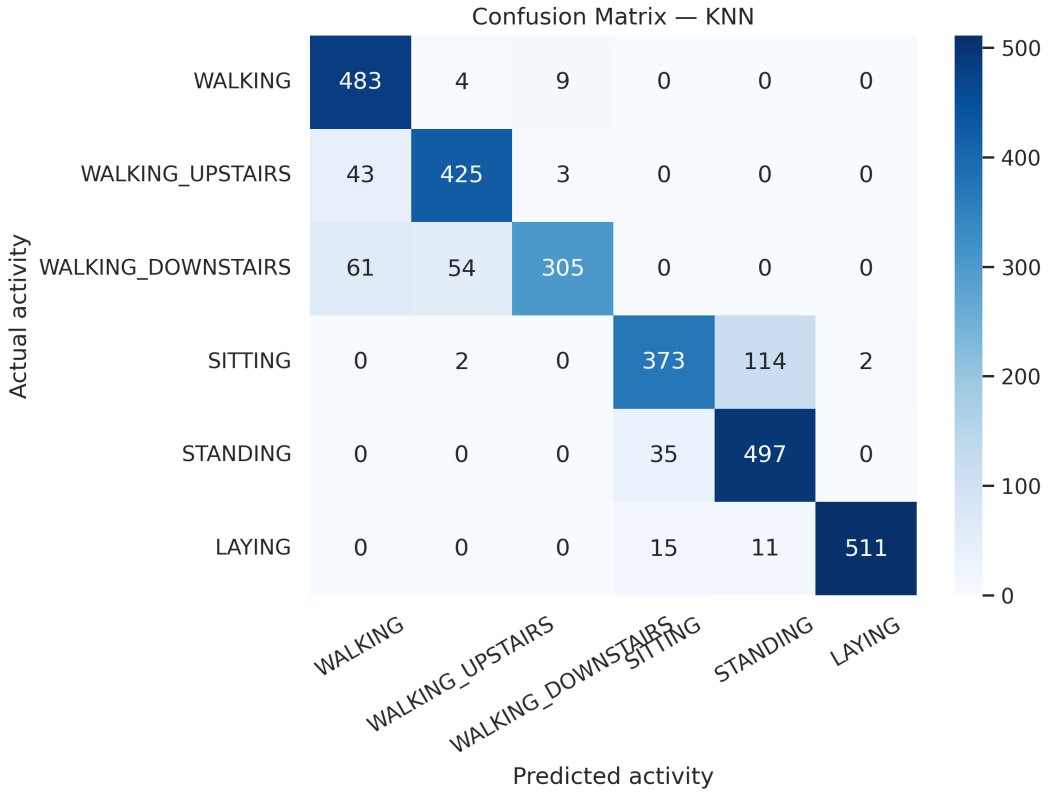


Figure 4.2: Confusion matrix for KNN.

For KNN, the model correctly identifies 483 of 496 walking observations and 511 of 537 laying observations. A notable confusion occurs between walking downstairs and walking: 61 walking-downstairs observations are predicted as walking. Sitting and standing are also confused in both directions.

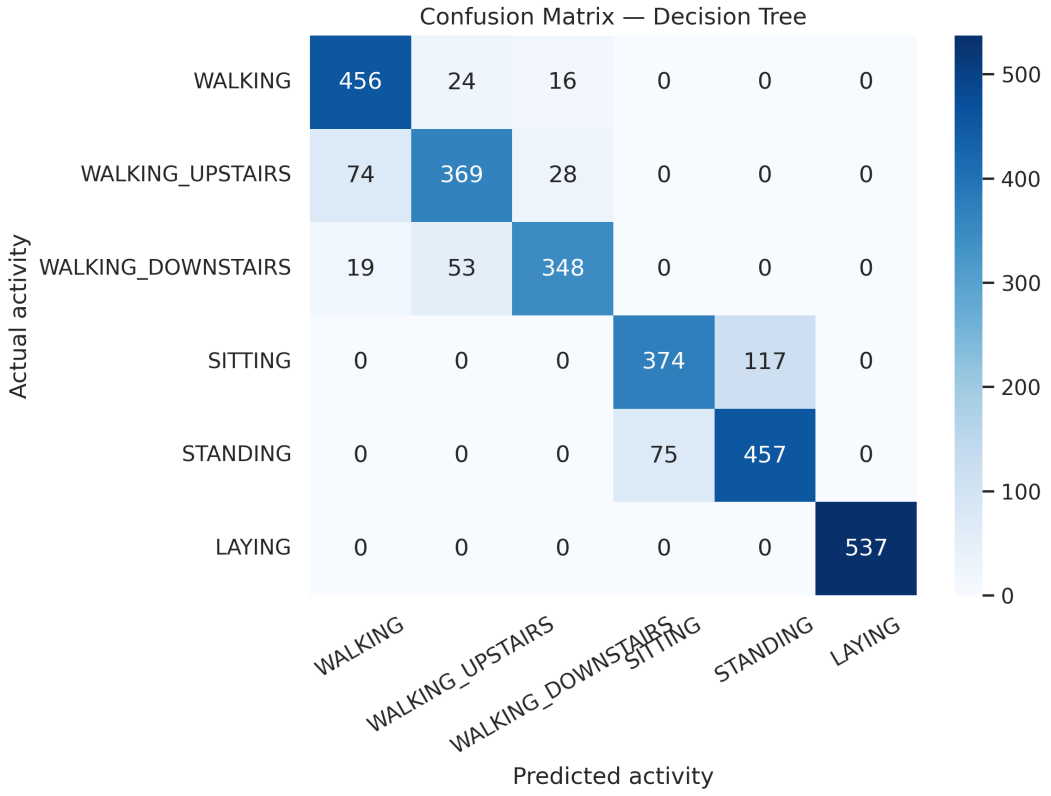


Figure 4.3: Confusion matrix for Decision Tree.

The Decision Tree correctly classifies all 537 laying test observations. However, it confuses sitting with standing: 117 sitting observations are predicted as standing, and 75 standing observations are predicted as sitting. Some confusion is also present among the three walking-related classes.

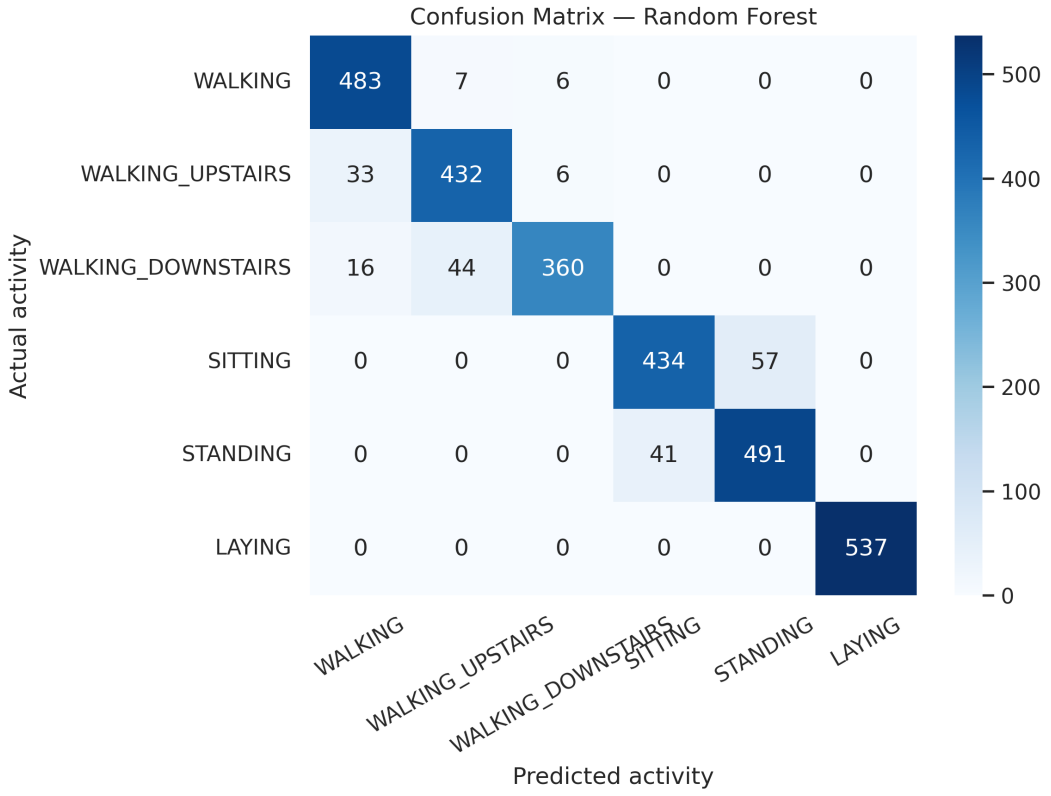


Figure 4.4: Confusion matrix for Random Forest.

Random Forest correctly classifies all 537 laying observations. It also improves the separation of sitting and standing compared with the other two models, although 57 sitting observations are predicted as standing and 41 standing observations are predicted as sitting. Among dynamic activities, the main remaining confusion is between walking and walking downstairs.

4.4 Class-wise Evaluation

The following tables provide class-wise precision, recall, F1-score and support. Support is the number of actual test observations belonging to the corresponding class. These tables provide more detail than overall accuracy alone.

4.4.1 Class-wise report: KNN

Table 4.2: Class-wise precision, recall and F1-score for KNN

Activity	Precision	Recall	F1-score	Support
WALKING	0.823	0.974	0.892	496
WALKING_UPSTAIRS	0.876	0.902	0.889	471
WALKING_DOWNSTAIRS	0.962	0.726	0.828	420
SITTING	0.882	0.760	0.816	491
STANDING	0.799	0.934	0.861	532
LAYING	0.996	0.952	0.973	537

4.4.2 Class-wise report: Decision Tree

Table 4.3: Class-wise precision, recall and F1-score for Decision Tree

Activity	Precision	Recall	F1-score	Support
WALKING	0.831	0.919	0.873	496
WALKING_UPSTAIRS	0.827	0.783	0.805	471
WALKING_DOWNSTAIRS	0.888	0.829	0.857	420
SITTING	0.833	0.762	0.796	491
STANDING	0.796	0.859	0.826	532
LAYING	1.000	1.000	1.000	537

4.4.3 Class-wise report: Random Forest

Table 4.4: Class-wise precision, recall and F1-score for Random Forest

Activity	Precision	Recall	F1-score	Support
WALKING	0.908	0.974	0.940	496
WALKING_UPSTAIRS	0.894	0.917	0.906	471
WALKING_DOWNSTAIRS	0.968	0.857	0.909	420
SITTING	0.914	0.884	0.899	491
STANDING	0.896	0.923	0.909	532
LAYING	1.000	1.000	1.000	537

4.5 Feature Importance

Random Forest provides an estimate of the relative importance of input features. Figure 4.5 shows the 20 features with the highest importance scores in the fitted model. The most important feature is `tGravityAcc-mean()-X`, with an importance score of approximately 0.03850. Other highly ranked features include gravity acceleration statistics and angle-related features.

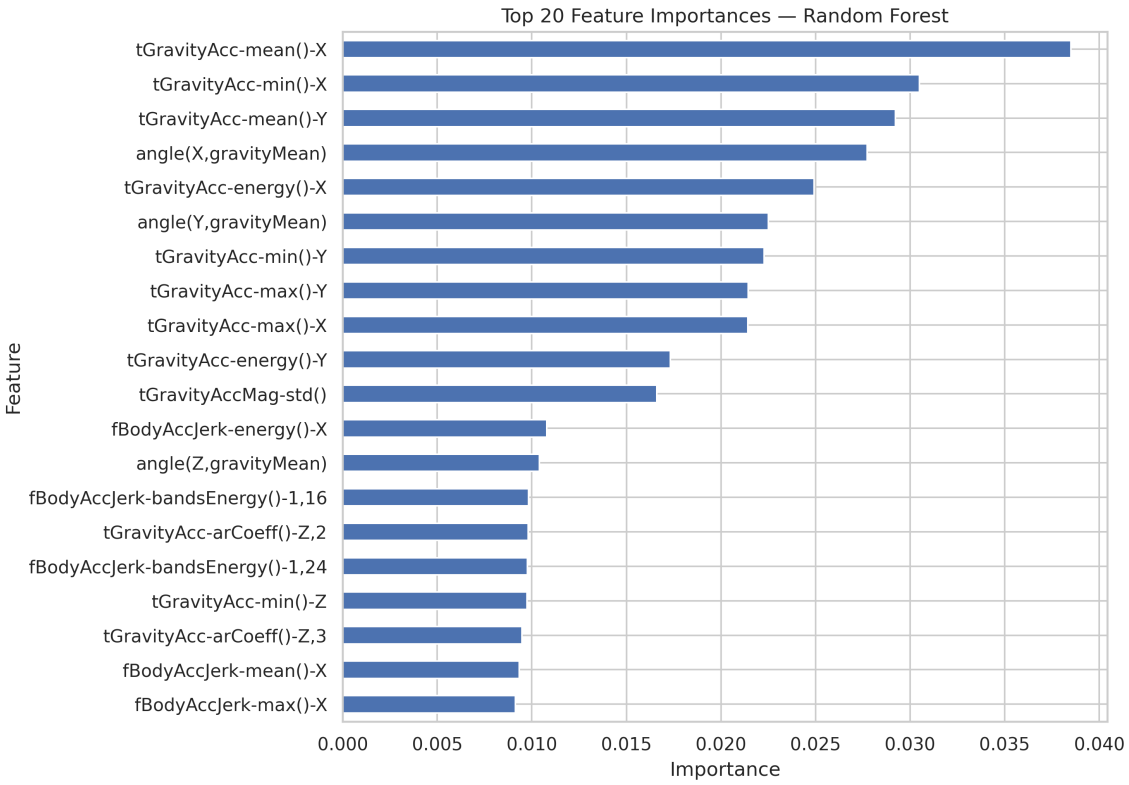


Figure 4.5: Top 20 features ranked by Random Forest feature importance.

Table 4.5: Top 10 Random Forest features by importance

Feature	**Importance**
tGravityAcc-mean()-X	0.03850
tGravityAcc-min()-X	0.03048
tGravityAcc-mean()-Y	0.02923
angle(X,gravityMean)	0.02773
tGravityAcc-energy()-X	0.02492
angle(Y,gravityMean)	0.02251
tGravityAcc-min()-Y	0.02228
tGravityAcc-max()-Y	0.02144
tGravityAcc-max()-X	0.02143
tGravityAcc-energy()-Y	0.01733

Feature importance indicates which variables contributed most to the fitted forest's decisions. It does not by itself establish a causal relationship between a sensor feature and a physical activity.

4.6 Single-Observation Prediction

The notebook demonstrates prediction for one test observation (sample index 0) using the Random Forest classifier. The actual activity is **STANDING**, and the predicted activity is also **STANDING**. This example demonstrates the inference call for one feature vector.

4.7 Best Model

Random Forest is selected as the best model because it achieves the highest macro F1-score (0.9270) and highest accuracy (0.9287) among the evaluated classifiers. It is therefore the model selected for the best-performing result in this experiment.

Chapter 5

Web Application, Discussion and Conclusion

5.1 Streamlit Web Application

A Streamlit web application is associated with this project and is available at:

https://human-activity-recognition-ehccfyncz87dfyderptkah.streamlit.app/

The notebook includes an export step that prepares files for the application. The exported artifacts include the fitted KNN, Decision Tree and Random Forest models in Joblib format, the test feature and label data in CSV format, a CSV file containing Random Forest feature importance, and a JSON metadata file. The metadata includes the feature names, activity mapping, evaluation results, best-model name and dataset dimensions.

Table 5.1: Files exported for the web application

Artifact	**Purpose recorded by the notebook**
`knn_har_model.pkl`	Saved KNN pipeline.
`decision_tree_har_model.pkl`	Saved Decision Tree classifier.
`random_forest_har_model.pkl`	Saved Random Forest classifier.
`X_test.csv`	Exported test feature matrix.
`y_test.csv`	Exported test activity labels.
`feature_importance.csv`	Random Forest feature importance values.
`metadata.json`	Feature names, class mapping, metrics, selected model and dataset dimensions.

5.1.1 Application Screenshots

Insert screenshots captured directly from the deployed Streamlit application in the `screenshots` folder. Suggested screenshots are the application landing page, the activity prediction interface, an example prediction result, and any model-performance or visualisation page that is actually present in the deployed application.

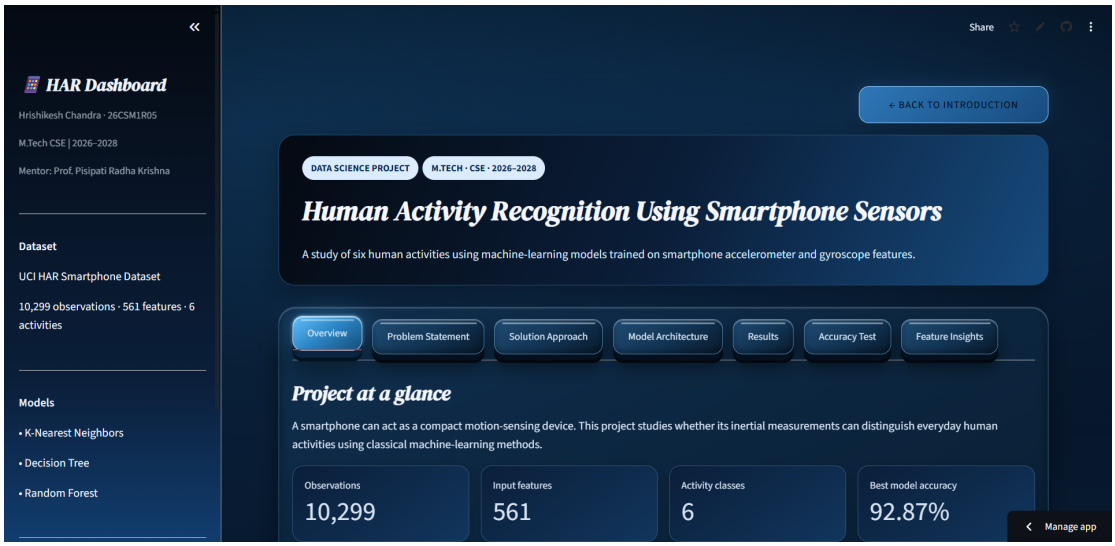


Figure 5.1: Streamlit application home page.



Figure 5.2: Example activity prediction in the web application.

Note: The application URL was supplied for this report, but its live page could not be

retrieved for independent inspection during report preparation. The screenshots and exact user-interface behaviour should therefore be confirmed using the deployed application before final submission.

5.2 Discussion

The experimental results show that ensemble learning provides the strongest performance among the three models tested on this feature dataset. Random Forest outperforms KNN and Decision Tree on all four reported aggregate metrics. Its macro F1-score of 92.70% indicates that its performance is strong when the six classes are weighted equally.

The confusion matrices show that laying is identified particularly well by the three classifiers. The most persistent ambiguity for the best model is between sitting and standing, and between walking and walking downstairs. This is reasonable in the context of sensor-based recognition because some activities can produce related movement or posture patterns in the engineered feature space. This interpretation is qualitative; the project does not include a separate causal or sensor-level investigation.

5.3 Limitations

- The notebook uses a prepared feature dataset rather than acquiring and processing live sensor streams.
- The project does not demonstrate deployment on a physical wearable device or microcontroller.
- The reported results are based on the supplied train/test partition; no additional cross-validation or external-dataset evaluation is shown in the notebook.
- The notebook evaluates three classifiers with the specified configurations. Further hyperparameter optimisation may change the results.
- The web application is linked in the report, but application screenshots and live behaviour must be verified by the project team before submission.
- The model predicts only the six activity categories represented in the training labels; it is not designed to identify arbitrary activities outside these classes.

5.4 Future Enhancements

Possible future extensions include collecting live accelerometer and gyroscope readings, implementing the documented filtering and feature-extraction pipeline on incoming sensor windows, evaluating the model on data from unseen users, optimising model size and inference latency, and extending the activity label set. The web application can also be enhanced with clearer prediction confidence information and a history of recognised activities, if these functions are required.

5.5 Conclusion

This project implemented a supervised machine-learning workflow for recognising six human activities from 561 engineered sensor features. The supplied dataset was loaded and checked for missing values, duplicate records and non-finite values. KNN, Decision Tree and Random Forest classifiers were trained and evaluated using the same test set of 2,947 observations. Random Forest achieved the best performance, with 92.87% accuracy, 93.00% macro precision, 92.58% macro recall and 92.70% macro F1-score. Confusion matrices and class-wise reports were used to study the prediction behaviour, while feature importance was used to identify influential variables. Finally, the notebook exported the trained models and supporting metadata for the associated Streamlit application. The work demonstrates how prepared inertial-sensor features and conventional machine-learning algorithms can be combined to build a practical six-class HAR classification workflow.

Bibliography

[1] UCI Machine Learning Repository, "Human Activity Recognition Using Smartphones Dataset." Available: https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones. Accessed: [Insert access date].

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] Scikit-learn Developers, "Scikit-learn User Guide: Supervised Learning." Available: https://scikit-learn.org/stable/supervised_learning.html. Accessed: [Insert access date].

[4] Streamlit, "Streamlit Documentation." Available: https://docs.streamlit.io/. Accessed: [Insert access date].

[5] Project implementation notebook and dataset documentation supplied for this report: `DSF_Model_Training.ipynb`, `features.txt`, `features_info.txt`, `activity_labels.txt`, and the corresponding training and testing data files.